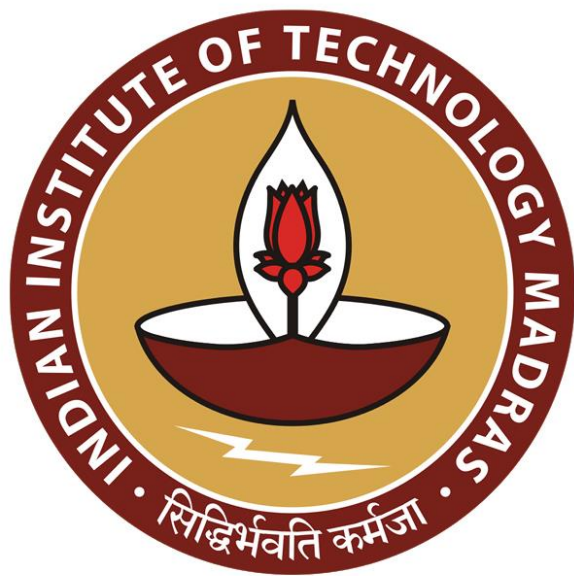




**IIT Madras**  
ONLINE DEGREE

**Programming Concepts Using Java**  
**Online Degree Programme**  
**B. Sc in Programming and Data Science**  
**Diploma Level**  
**Prof. Madhavan Mukund**  
**Department of Computer Science**  
**Chennai Mathematical Institute**

**Lecture 3**  
**Memory Management**

So, we saw that variables have types which determine what we can do with them. Now let us look at how variables are actually stored during the execution of our program.

(Refer Slide Time: 00:25)

**Keeping track of variables**

- Variables store intermediate values during computation
  - Typically these are local to a function
  - Can also refer to global variables outside the function
  - Dynamically created data, like nodes in a list
- **Scope of a variable**
  - When the variable is available for use
  - In the following code, the `x` in `f()` is not in scope within call to `g()`
- **Lifetime of a variable**
  - How long the storage remains allocated
  - Above, lifetime of `x` in `f()` is till `f()` exits
  - "Hole in scope" — variable is alive but not in scope

```
def f(l):  
    ...  
    for x in l:  
        y = y + g(x)  
    ...  
def g(m):  
    ...  
    for x in range(m):  
        ...
```

So, normally we have variables to store intermediate values during our computation, and they come in three broad categories. The first are variables that are local to the function in which they are declared. So, these are for instance local variables that you use for iteration for `x` and `l` in python program. So, `x` would be a variable which goes over the elements in the list, and you do not want this `x` to clash with any other `x`.

Then you may refer in this function to variables which are declared outside. So, so-called global variables which are declared in enclosing blocks which call this function, and finally you might have dynamically created data like a linked list where you add nodes. So, you only have a variable which points to the first element in the list but the internal elements which are pointed to by internal pointers the next pointers in the list are also part of the storage that your program uses.

So, the first thing that we need to understand is when a variable is meaningful in a program. So, when is a variable available for our program to refer to? So, this is called the scope of a variable. So, let us look at this code. So, we have a function `f` on the left which uses a local variable `x` and in the course of computing that loop it calls another function `g`, and `g` in turn uses another `x` in order to range over the numbers 0 to  $m - 1$ .

So, clearly the `x` on the right is not the same as the `x` on the left. So, the `x` in `f` even when `f` is executing temporarily goes out of scopes that is when we go from here to `g` and back when we go into `g` the `x` which is defined in `f` is no longer in scope it is no longer available for use because there is another `x` inside `g` which blocks it or masks it. On the other hand, the `x` which is there in `f` does not disappear obviously because we have to then go back in the next iteration and go to the next element of the list.

So, there is storage allocated for `x` when `f` starts and this storage will persist until `f` ends, even though in between that `x` is not available to us when we go to `g` and come back. So, this is called the lifetime of variable. So, the lifetime of a variable tells us from the time that we allocate space for the variable how long that space is available in memory without getting destroyed.

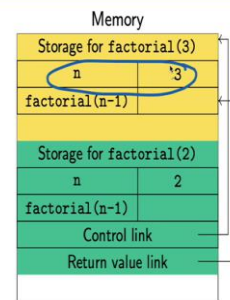
So, the lifetime can as we saw in this example exceed the scope there may be periods when the lifetime of the variable is valid that is there is still storage for that variable but because of the way our code is written it is not in scope. So, this is sometimes called a hole in the scope right. So, there is a period during the lifetime of a variable when we cannot actually access it because it is not available to us.

**(Refer Slide Time: 03:10)**



## Memory stack

- Each function needs storage for local variables
- Create **activation** record when function is called
- Activation records are stacked
  - Popped when function exits
  - Control link points to start of previous record
  - Return value link tells where to store result
- Scope** of a variable
  - Variable in activation record at top of stack
  - Access global variables by following control links
- Lifetime** of a variable
  - Storage allocated is still on the stack



- Call **factorial(3)**
- factorial(3)** calls **factorial(2)**



Madhavan Mulu

Memory Management

Programming Concepts using Java 3/7

So, how do we now manage these different kinds of variables with respect to the memory so that we have no ambiguity about which variable we are referring to and which variable is in scope. So, remember that each function needs storage separately for its local variables we need an x to be stored for f in the previous example and a different x to be stored for g.

The way this is usually maintained is to provide space for the variables of each function in what is called an activation record. So, let us consider our usual definition of factorial the recursive definition of factorial which says factorial of n is 1 if n is 0 and otherwise it is n times factorial of n - 1 the typical recursive definition. So, if you want you can write def factorial n so you can say if n is equal to 0 return 1 else return n times factorial of n - 1.

So, this is the typical function that we are going to deal with. So, inside this function there is a variable n right namely the parameter. So, here is the activation record when we call factorial of 3. So, factorial of 3 will have n set to 3. So, that is what this line is saying and it is going to eventually need the value of factorial n - 1 but I do not know what that value is until I execute that function.

So, I have a space to store factorial of n minus 1 in my activation record but at this point I do not have a value for this. So, now factorial of 3 is going to call factorial of 2. So, this will create a new activation record in which factorial of 2 has its value n equal to 2 and 2 in turn is going to call factorial of 1 for which it does not have a space. Now what happens eventually is a factorial of 1 will call factorial of 0 and factorial 0 will return 1 and so on.

So, how does that process work so, when the function exits this will go off the stack. So, this activation record will be removed because factorial 2 is no longer active but different functions may have different amount of space. So, we do not know in advance what is the size of an activation record. So, we need to know where the previous activation record is.

So, this is maintained in each activation record through what is called a control link. So, when I create an activation record for factorial 2 I also add this link which tells me where the previous activation record from where this function was called was begins. So, that when I go back one step when I finish factorial of 2 I know now what is the beginning of the activation record that is on top of the stack.

The other thing is that we need to populate the space right factorial of  $n - 1$  and this value comes back from the function which just exited. So, I have a return value link right a return value link tells me where to take the return value that I produce in this function and store it in the previous activation record. So, therefore an activation record contains space for the local variables.

And then for book keeping you have to keep track of where the previous activation record was so, that you can restore it when you exit and also where to return the value that you are going to return from now and store it in the previous activation record because usually it will be stored somewhere in the calling function. So, this is how this memory stack works.

So, now with respect to this stack we can talk about the scope of a variable. So, the scope of a variable basically says that if it is local to a function it will be the top most function. So, if I am talking about  $n$  for instance at this stage when I have called factorial of 2 then the scope of  $n$  is this activation record. So, I look at the top of the stack. Now if I had a global variable in this case I do not.

But supposing there was a reference to  $x$  in this function then and I do not find an  $x$  in the local activation record I would then go back and say ok let me see if there is an  $x$  in the function that called me and if there is no  $x$  there then typically here there will be another control link and you will go back to the function that called factorial of 3 and so on. So, you will keep going outside the scope in some sense there is an enclosing scope in order to find it.

So, that is how you can keep track of both global and local variables. So, typically in a python kind of a situation where you have a main function which starts then those values will be at the bottom of the stack and they will be reached by a sequence of links from whatever function is currently active. And in this case the lifetime of a variable is determined by whether or not it remains on the stack.

So, the lifetime of this n equal to 3 is as long as this activation record for factorial 3 remains on the stack. Now if at a later point factorial of 3 exits and it is called again that will be a fresh factorial 3 activation record and a fresh n. So, that will not be in the same lifetime as this n.

**(Refer Slide Time: 08:07)**

**Passing arguments to functions**

- When a function is called, arguments are substituted for formal parameters

```
def f(a,l):
    ...
    x = 7
    myl = [8,9,10]
    f(x,myl)
```

*Handwritten annotations: Blue arrows point from 'x = 7' to 'a = x' and from 'myl = [8,9,10]' to 'l = myl'. A box labeled 'l=' is also present.*

- Parameters are part of the activation record of the function
  - Values are populated on function call
  - Like having implicit assignment statements at the start of the function
- Two ways to initialize the parameters
  - Call by **value** — copy the value
    - Updating the value inside the function has no **side-effect**
  - Call by **reference** — parameter points to same location as argument
    - Can have **side-effects**
    - Be careful: can update the contents, but cannot change the reference itself

*Video inset: A man in a plaid shirt.*

IIT Madras BSc Degree

Mathavan Mukund Memory Management Programming Concepts using Java 4/7

So, one thing we mentioned there is that n was actually stored on the stack as an argument to factorial. So, when a function is called the arguments are actually substituted for the formal parameters in the function definition. So, if I have some function like this for instance f which has two arguments a and l and then I set up some code where I set x equal to 7 and my list mi l equal to 8 9 10 and then I call it then what this means is that x is going to play the role of a in the function and my l is going to play the role of l in the function.

So, I have to substitute the values that I pass as arguments for the formal parameters in the function. So, as we saw before these parameters are part of the activation record they are populated when the function is called and I create the activation record these values get initialized. So, I can actually think of it as having an implicit initialization I can say that I have like these two lines at the beginning of my function which populate these values in my activation record.



So, it is as though I set a equal to x and then I set l equal to my l and then I run this code. So, it is although almost as I have a copy of that code with these initializations as assignment statements at the top of my function. Now there are two ways in which these values can be passed from the argument to the formal parameter. So, the first one is called call by value. So, here I just copy the value.

So, here I would take this 7 and I would say that I would assign a fresh location and call that fresh location having the same value 7. This is what happens in python and what are called immutable variables right when I do an immutable assignment in python I effectively get two copies of the values. So, that if I update one value the other one does not get changed.

On the other hand I could have what is called call by reference. So, in a call by reference situation so, this my list is pointing at this point to some location which has 8 9 10 and now when I say l is equal to my list this is also pointing to the same location. So, I am pointing the parameter variable to the same location as the argument variable. Therefore if I make a change to l inside the function it also makes a change to the my list outside the function.

So, in this situation you can have what are called side effects right a function can update something inside which will have impact outside. And this is usually used for large structures like lists and dictionaries where a it is inefficient to copy everything and make a fresh copy of the whole list of the dictionary inside the function and b very often you do want side effects like you want a function that can sort a list or you want a function that can reverse a list.

So, you would like the resulting list to have that property that you have executed inside the function. Now one thing one has to be careful about is that one cannot change the reference one can change what the reference is pointing to. So, I can take this list and I can update it in place right I can make this into say 9 9 10 by replacing the first value. But if I take this l and make it point to another thing I create a fresh list by doing an l equal to something inside my code inside the function code this will actually change what l is pointing to and now what the list outside and the list inside are pointing to are no longer the same.

So, this is something that one has to be wary of when passing by reference. So, you can pass by reference but if you expect the change to be reflected as a side effect then inside you can

only make so, called what python calls mutable changes right you can only update in place but you cannot reassign that reference to point to something else. A typical example of this happens when you want to insert a value at the beginning of a list.

So, you cannot make the head of the list point to the new node if you remember you have to make the new node in some sense you have a list and you have this. So, you will create a new node and then you will do some plumbing. So, you will copy the new value here you will make this the old value and then you will basically root your list like this. So, these are the kinds of things that you have to be a little careful of when you do call by reference.

(Refer Slide Time: 12:12)

**Heap**

- Function that inserts a value in a linked list
  - Storage for new node allocated inside function
  - Node should persist after function exits
  - Cannot be allocated within activation record
- Separate storage for persistent data
  - Dynamically allocated vs statically declared
  - Usually called the heap
    - Not the same as the heap data structure!
  - Conceptually, allocate heap storage from "opposite" end with respect to stack
- Heap storage outlives activation record
  - Access through some variable that is in scope

**Memory**

Stack

Heap

Madhavan Mukund

Memory Management

Programming Concepts using Java 5/7

So, speaking about lists what happens to the values which are created by functions which do things like insert a node in a list. So, clearly the storage for this new node is created when the function runs but if it is part of the stack then it will also be destroyed when the function exists and this is not what we intend right. So, we intend that this node that we create inside the function that adds a node to a list should persist after the function exits.

So, therefore it cannot be on the activation record because if it were on the activation record then it will go as soon as the function pops. So, this kind of data is called dynamically allocated. So, this is data which we create not as part of the names that are there in our program. So, statically declared data consists of all the variables that we actually use in our code.

So, in a language like python statically declared means that we actually name the variable and assign something to it in other languages like java it means that we actually declare it up front



say this int is i is an int or x is a float and so. On so, these statically declared variables will be defined on the stack but through these statically declared variables I can create new storage which is dynamic it is created as the function runs but it does not have a name of itself.

It is accessed through a statically declared name. So, this storage is usually called the heap this is a separate piece of storage from the stack is called the heap traditionally. Now one thing to be careful is that this has nothing to do with the heap data structure that we have studied in the data structures course. So, this is not to do with priority queues or anything it is a disjoint storage from the stack and it has no particular structure it does not like the stack has this last in first out structure the heap has no particular structure. So, conceptually you want to store the heap disjoint from the stack.

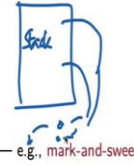
So, our stack remember is growing downwards the way we have assumed. So, we have memory and the memory is a sequence of locations and the stack is growing from one end. So, what we do is we allocate space from the opposite end for the heap. So, that the heap will not intersect with the stack. So, we will not have any confusion that the stack kind of overruns the heap at some point.

So, the main point is that the storage in this location whatever is inside the heap does not get deallocated. So, the stack will basically. Now the top of the function the last function if it pops then this part of the stack will be removed from the memory. So, that for those variables will no longer exist but remember that even though the heap persists in memory we need to have a way of getting at it.

So, we would need to know for instance if there is a list in memory here which has been stored we would need to have an access to it through a variable on the stack. So, if we do not have any variable on the stack which points to the element of the heap directly or indirectly then that variable on the heap is effectively unusable for us.

**(Refer Slide Time: 15:01)**

- On the stack, variables are deallocated when a function exits
- How do we "return" unused storage on the heap?
  - After deleting a node in a linked list, deleted node is now **dead** storage, unreachable
- Manual memory management
  - Programmer explicitly requests and returns heap storage
    - `p = malloc(...)` and `free(p)` in C
  - Error-prone — memory leaks, invalid assignments
- Automatic garbage collection (Java, Python, ...)
  - Run-time environment checks and cleans up dead storage — e.g., **mark-and-sweep**
    - Mark all storage that is reachable from program variables
    - Return all unmarked memory cells to free space
  - Convenience for programmer vs performance penalty



Madhavan Melund

Memory Management

Programming Concepts using Java 6/7

So this leads us to another question. So, when we had a stack the memory management was easy when I create a function when I call a function I create an activation record put it on the stack when the function exits I remove it from the stack and correspondingly all the storage that was allocated for that function disappears. So, there is no problem it is a clean system where I know when I add the memory and I know when I remove the memory.

Now in a dynamically created data point like we saw it is created in and put into the heap and we have to know that we have an access to it otherwise we would not know whether this part of the heap is in use or not. So, periodically of course we do also want to get rid of data from the heap. So, for instance if I have a delete node function right then the deleted node is now dead storage because I would typically have a list in which I have some links right and then I want to delete this node and the way I delete it is by making this point directly to the node after it.

So, I bypass the node in the link structure. So, now there is no access to this deleted node anymore from my variables in my program. On the other hand at some point I had taken the storage by asking for it. So, I need a way of telling the system that this storage is no longer being used. So, how do we do that? So, in some languages like c for instance you do this manually.

When you ask for storage you call a function and then when you return the storage you call another function which frees that location. So, you have to make sure that you free everything

that you do not use and if you do not free it right then that storage will be assumed to be in use and this leads to what is called a memory leak. So, you should think of your memory as a tank.

So, think of the free space as a tank of water right and every time you ask for something you open a tap and it gives you some water. So, your water is your storage space. So, memory leak corresponds to saying that your storage phase is getting emptied out without being replenished by being restored. So, if you do not restore free space to your memory then eventually your memory runs out and you have a memory leak this leads to obviously performance penalties because system slows down in the worst case your system can actually grind to a halt.

This does happen with large systems which run for a long time for example browsers even operating systems which are not written properly can start getting resource hungry as they run and then sometimes you have to restart in order to restore this the performance to its usual level. The other thing that can happen with this manual memory management is that you have freed  $p$  by doing this but maybe you kind of forgot or your code is such that there may be situations after this where you believe that  $p$  is still pointing to something valid and you make an assignment.

Now depending on how type safe your language is that assignment may or may not be allowed and if it is allowed then it may be that you assign some value to a random piece of memory which you do not have access to. So, therefore working with explicit memory management can be error prone and it is difficult very often to keep track of things systematically especially large bodies of code.

So, the alternative to memory management being done manually is obviously to do it automatically and this is what is called garbage collection. So, languages like java and python actually use automatic garbage collection and what this means is that they periodically check that storage is not being used and return it to the free space. So, one common way to do this is what is called mark and sweep.

So, this is a little bit like a reachability check in our graph. So, we have at any given point if our program is running. So, on the stack right we have a bunch of variables which our program is currently using and they are all pointing to different places in the heap. So, what we do is we go through this stack and we access every place on the heap that is reachable from something

on the stack and we mark it at the end of it anything which is on the heap which is not marked is not reachable from the stack.

So, we can assume it is free space. So, all the unmarked cells can be returned back to the free space then we can resume the program. So, this is a simple way to do automatic garbage collection. Now the problem with this of course is that this has to be done while you are running the program periodically. So, this garbage collector will run in parallel with your actual code.

So this means that when you are running java or python code there could be unexpected slowdowns in your code because your code has to be suspended. Remember that it is not very reliable to be running your code and doing garbage collection at the same time because there could be some contention you might be asking for some data and the garbage collector may have marked it as free and then there will be some inconsistency.

So, having what is called a concurrent garbage collector which runs while your program is running is a bit tricky. So, you actually have to stop the program and then run the garbage collector and this means that your program would unexpectedly run slower because the garbage collector ran at that particular time and the next time it went faster. So, the performance can be erratic.

So, you have the advantage that the programmer does not have to keep track of this memory explicitly but you have the disadvantage that the program performance is subject to these slowdowns which are not within your control.

**(Refer Slide Time: 20:16)**

- Variables have **scope** and **lifetime**
  - Scope — whether the variable is available in the program
  - Lifetime — whether the storage is still allocated
- Activation records for functions are maintained as a stack
  - Control link points to previous activation record
  - Return value link tells where to store result
- Heap is used to store dynamically allocated data
  - Outlives activation record of function that created the storage
  - Need to be careful about deallocating heap storage
  - Explicit deallocation vs automatic garbage collection



Madhavan Mooland

Memory Management

Programming Concepts using Java

7/7

So, to summarize what we have seen is that in order to manage our memory we have to talk about how this variables are stored in memory. So, variables have a scope which is how long the storage allocated to them is available to us on the in the memory and a lifetime sorry scope tells us whether we can access the variable or not and a lifetime which tells us how long the storage persists.

So, it could be that because you have called a function which hides a variable a variable might still be alive in the sense that storage may still be on the stack but it may not be in scope. So, for functions and local variables declared in functions we have these activation records which are stored on a stack. So, every time you call the function you put an activation record on the stack every time you exit a function you pop it.

And for this you also keep these control links and return value links to tell you where to go back after popping it and where to return the value that you just created and for values which are defined dynamically like new nodes in a list and so on. So, these are not values which are pointed to by variables on the stack and also they should outlive the stack the function that creates a list should not destroy the node after it exits.

So, these dynamic storage area data values are stored in what is called the heap which is a disjoint piece of memory but here we have to be careful about how to restore free space back from the heap. If we do it manually we could have some problems with memory leaks and other issues. On the other hand we could use automatic garbage collection which is what java



and python use which is convenient for the programmer but it might have performance penalties.

